

TRABAJO PRÁCTICO

Caso de estudio — Gestión de Residuos Urbanos

Alumnos:

Pepino, Facundo

Truchet, Mauricio

Docentes:

Ing. Bono, Juan Pablo

Asignatura:

Arquitecturas Móviles

Ciclo Lectivo:

2026

ÍNDICE

1. CONSIGNA 3
2. INTRODUCCIÓN 4
3. REQUERIMIENTOS DEL SISTEMA 5
3.1. Requerimientos funcionales 5
3.2. Alternativas de desarrollo móvil evaluadas 6
3.3. Requerimientos no funcionales 6
4. DISEÑO DE ARQUITECTURA 7
4.1. Estructura general de la solución 7
4.2. Flujo de trabajo y secuencia operativa 8
4.3. Modelo de persistencia y datos en Cloud Firestore 10
4.4. Algoritmo de optimización de rutas de recolección 10
4.5. Sensores en los contenedores y protocolo de comunicación 11
5. PROTOTIPO FUNCIONAL 12
5.1. Módulo de autenticación, perfiles y control de acceso 12
5.2. Módulo ciudadano 13
5.3. Panel de control para autoridades municipales 14
5.4. Módulo operativo para personal de cuadrilla 14
5.5. Plataforma de cartografía y geolocalización 15
5.6. Capturas de la interfaz en ejecución 15
6. ESQUEMA DE ESCALABILIDAD Y MONITOREO 16
6.1. Contenerización de servicios con Docker 16
6.2. Orquestación y balanceo con Kubernetes 16
6.3. Autoescalado horizontal de réplicas 17
6.4. Observabilidad y monitoreo con Prometheus y Grafana 18
7. ESTRATEGIA DE PRUEBAS Y CALIDAD 19
8. PROBLEMAS ENCONTRADOS Y SOLUCIONES TÉCNICAS 20
9. ENTREGABLES Y CUMPLIMIENTO DE CONSIGNAS 21
10. INNOVACIÓN Y PARTICIPACIÓN CIUDADANA 23
11. CONCLUSIONES 23
12. BIBLIOGRAFÍA 23

1. CONSIGNA

Caso de estudio Web Mobile App Gestión Residuos. Trabajo práctico de desarrollo de un sistema de gestión de residuos vecinales para la ciudad de San Francisco.

Contexto.

La ciudad de San Francisco necesita implementar un sistema de gestión de residuos vecinales que permita optimizar la recolección de desechos, mejorar el control de residuos especiales (reciclables, voluminosos, peligrosos) y fomentar la participación ciudadana en el cuidado ambiental.

La solución debe ser eficiente y escalable, involucrando tanto a los ciudadanos como a las autoridades locales y al personal de recolección.

Objetivo del trabajo práctico.

El objetivo de este trabajo es diseñar y prototipar un sistema de aplicaciones móviles y servicios en la nube para gestionar la recolección de residuos en la ciudad de San Francisco.

El sistema deberá contar con aplicaciones específicas para los ciudadanos, autoridades y personal de recolección, utilizando herramientas de escalado y optimización en tiempo real para satisfacer la demanda de una ciudad en crecimiento.

Requerimientos.

- Aplicación ciudadana. Registro de residuos generados (categoría y cantidad). Solicitud de recolección para residuos especiales (reciclables, voluminosos, peligrosos). Acceso a la ubicación de puntos de reciclaje y horarios de recolección. Recepción de notificaciones sobre campañas de reciclaje o eventos de limpieza.
- Aplicación de autoridades. Visualización de los puntos de residuos a recolectar y su estado en tiempo real. Planificación y asignación de rutas de recolección para el personal. Monitoreo del estado de los puntos de recolección y gestión de incidentes.
- Aplicación para el personal de recolección. Visualización de las rutas de recolección asignadas y confirmación de tareas completadas. Escaneo de códigos QR o ingreso de identificadores para registrar la recolección en puntos específicos. Reportes sobre el estado de los contenedores o incidentes en el servicio.
- Plataforma de mapeo y geolocalización. Integración con una API de geolocalización (Google Maps, Mapbox, etc.) para localizar los puntos de recolección y generar rutas optimizadas en tiempo real.
- Notificaciones y escalabilidad. Implementar un sistema de notificaciones push para ciudadanos y personal de recolección. Diseñar la arquitectura para ser escalable

usando contenedores (Docker) y orquestación (Kubernetes). Utilizar herramientas de monitoreo y autoescalado para asegurar el rendimiento y la disponibilidad del sistema bajo distintas cargas de uso.

Entregables.

- Diagrama de arquitectura. Un diagrama detallado que describa la arquitectura de la solución, mostrando la interacción entre los diferentes componentes (frontend, backend, bases de datos, APIs de geolocalización, y servicios de escalado).
- Prototipo funcional. Prototipo de la aplicación ciudadana, que permita probar el flujo de registro y solicitud de recolección. Prototipo de la aplicación para autoridades y personal de recolección, con funcionalidades básicas de monitoreo y asignación de tareas.
- Documentación técnica. Explicación detallada de las decisiones arquitectónicas. Justificación de las herramientas y servicios de escalado elegidos (Docker, Kubernetes, etc.). Descripción del flujo de trabajo y las rutas optimizadas de recolección de residuos.
- Esquema de escalado y monitoreo. Describir las estrategias de escalabilidad y monitoreo implementadas (uso de balanceadores de carga, autoescalado, monitoreo con Grafana/Prometheus o similar).

Criterios de evaluación.

- Diseño de arquitectura. Claridad, escalabilidad, y justificación de las herramientas seleccionadas.
- Funcionalidad del prototipo. Cumplimiento de las funcionalidades especificadas para cada rol.
- Documentación técnica. Explicación y justificación clara de cada componente y flujo de trabajo.
- Innovación y creatividad. Propuestas para mejorar la eficiencia del sistema o ideas adicionales para aumentar la participación ciudadana.

2. INTRODUCCIÓN

La gestión de residuos de San Francisco, en la provincia de Córdoba, enfrenta barrios cada vez más dispersos y un servicio de recolección que todavía se organiza de forma manual. La solución propone una plataforma digital única, en la que vecinos, municipio y cuadrillas trabajan sobre la misma información y en tiempo real.

La plataforma atiende a tres actores.

- Los vecinos clasifican sus residuos, piden el retiro gratuito de materiales especiales y consultan los ecopuntos más cercanos, sus horarios y las campañas vigentes.

- Las autoridades municipales supervisan el estado de los contenedores, reparten el trabajo entre las cuadrillas y resuelven los reclamos.
- El personal de recolección recorre rutas optimizadas, confirma cada parada con un código QR y reporta problemas en la calle al instante.

La arquitectura combina una aplicación web para celulares con servicios de nube de Google y un despliegue en contenedores sobre Kubernetes. De ese modo el sistema suma recursos de forma automática ante picos de demanda, como los que siguen a una tormenta o a una campaña de descacharrado.

3. REQUERIMIENTOS DEL SISTEMA

3.1. Requerimientos funcionales

El siguiente cuadro detalla las capacidades que debe ofrecer el sistema a cada perfil de usuario.

Identificador	Perfil	Descripción funcional	Prioridad
RF-01	Vecino	Registrar los residuos que genera indicando categoría y cantidad estimada	Alta
RF-02	Vecino	Solicitar el retiro programado de residuos reciclables, voluminosos y peligrosos	Crítica
RF-03	Vecino	Ver en un mapa los ecopuntos con su dirección, horario de atención y nivel de llenado	Alta
RF-04	Vecino	Recibir avisos en el celular sobre campañas ambientales y avances de sus pedidos	Media
RF-05	Autoridad	Seguir en tiempo real las solicitudes pendientes, en curso y recolectadas	Crítica
RF-06	Autoridad	Calcular la ruta más corta para las solicitudes elegidas y asignarla a una cuadrilla	Crítica
RF-07	Autoridad	Atender los incidentes reportados por vecinos y choferes hasta su resolución	Alta
RF-08	Autoridad	Enviar avisos de concientización a todos los vecinos	Media
RF-09	Cuadrilla	Consultar la hoja de ruta con las paradas del turno, ordenadas y trazadas por las calles	Crítica
RF-10	Cuadrilla	Validar cada contenedor escaneando su código QR	Alta
RF-11	Cuadrilla	Confirmar el peso recolectado y registrar observaciones de la parada	Alta

Identificador	Perfil	Descripción funcional	Prioridad
RF-12	Cuadrilla	Reportar problemas en la vía pública, como contenedores dañados o calles cortadas	Alta
RF-13	Autoridad	Supervisar el llenado de los contenedores medido por sensores y recibir alertas de desborde	Crítica
RF-14	Seguridad	Identificar a cada persona con su cuenta de Google y limitar lo que puede hacer según su rol	Crítica

Requerimientos funcionales por perfil y prioridad.

3.2. Alternativas de desarrollo móvil evaluadas

Se compararon tres formas de construir la aplicación según su costo, el tiempo hasta la puesta en marcha y la experiencia de uso. La aplicación web instalable cubre todos los requerimientos desde el navegador.

El teléfono aporta la ubicación del camión y la cámara que lee los códigos QR. El service worker, un programa que el navegador mantiene en segundo plano, permite abrir la aplicación sin cobertura y recibir avisos.

Alternativa	Ventajas	Desventajas	Decisión
Aplicación web instalable (PWA)	Se publica al instante sin pasar por las tiendas, funciona sin conexión y consume pocos datos	Acceso más limitado a algunos componentes del teléfono	Elegida por llegar a cualquier vecino con bajo costo de mantenimiento
Aplicación híbrida (Capacitor o Cordova)	Se instala desde la tienda y accede a funciones del teléfono mediante puentes	Compilación más compleja y demoras de aprobación en las tiendas	Descartada porque exige descargar la aplicación para trámites ocasionales
Aplicación nativa (Kotlin o Swift)	Mejor rendimiento en animaciones y acceso directo al sistema del teléfono	Hay que mantener un código distinto para cada sistema, con un costo de desarrollo alto	Descartada por presupuesto y plazos

Comparación de las alternativas para construir la aplicación.

3.3. Requerimientos no funcionales

Los atributos de calidad condicionan la estructura del software y las decisiones de infraestructura.

- Disponibilidad. El servicio debe estar activo el 99,9% del año, es decir menos de nueve horas de interrupción acumulada.

- Rapidez. El 95% de las consultas y los registros debe responder en menos de 250 milisegundos.
- Escalabilidad. La infraestructura suma réplicas automáticamente, de 2 a 10, cuando el uso del procesador supera el 70%.
- Tiempo real. Los cambios de estado de una solicitud aparecen en los paneles conectados en menos de un segundo.
- Cobertura irregular. La aplicación se abre sin conexión y sincroniza los datos al recuperar la señal.
- Seguridad. Toda la comunicación viaja cifrada y el servidor exige los permisos de cada rol.
- Uso en celulares. Las pantallas son táctiles, desde 360 píxeles de ancho, y cumplen criterios de accesibilidad.
- Livianidad. La primera carga ocupa menos de 2 megabytes para que sea ágil en redes celulares.
- Sensores. Las lecturas de llenado se reciben en menos de 500 milisegundos con entrega confirmada.

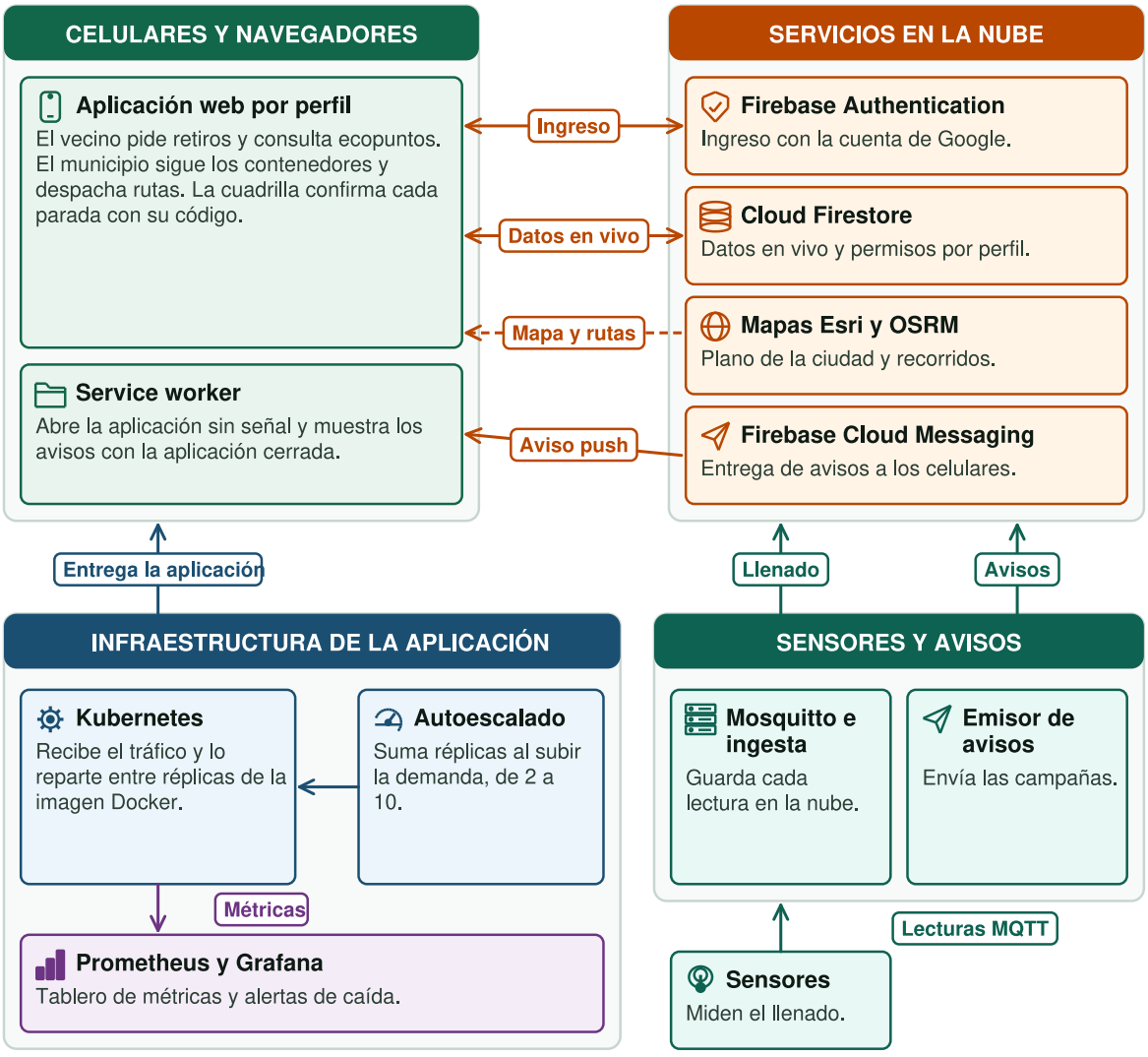
4. DISEÑO DE ARQUITECTURA

4.1. Estructura general de la solución

La arquitectura se organiza en cuatro bloques. Cada uno cumple una función propia y puede cambiar sin afectar a los demás.

- Los celulares y navegadores ejecutan la aplicación web. Un service worker, un programa que el navegador mantiene en segundo plano, la abre sin conexión y muestra los avisos aunque la aplicación esté cerrada.
- Los servicios de Google identifican a cada persona con su cuenta, guardan los datos en tiempo real con Cloud Firestore y entregan los avisos con Firebase Cloud Messaging. El mapa usa teselas de Esri y los recorridos por calles provienen de OSRM.
- La infraestructura de la aplicación empaqueta el sitio en una imagen Docker. Kubernetes reparte el tráfico entre las réplicas, el autoescalado ajusta su cantidad y Prometheus y Grafana vigilan su estado.
- Los sensores de los contenedores envían sus lecturas a Mosquitto, el intermediario de mensajes, y un servicio de ingesta las guarda en Firestore. Un emisor envía las campañas a los celulares registrados.

El prototipo se publica en Firebase Hosting, que distribuye los archivos desde servidores cercanos a cada usuario. La imagen Docker y Kubernetes llevan el mismo servicio a una infraestructura propia del municipio, junto con Mosquitto y la ingesta, y se verificaron en un clúster local.



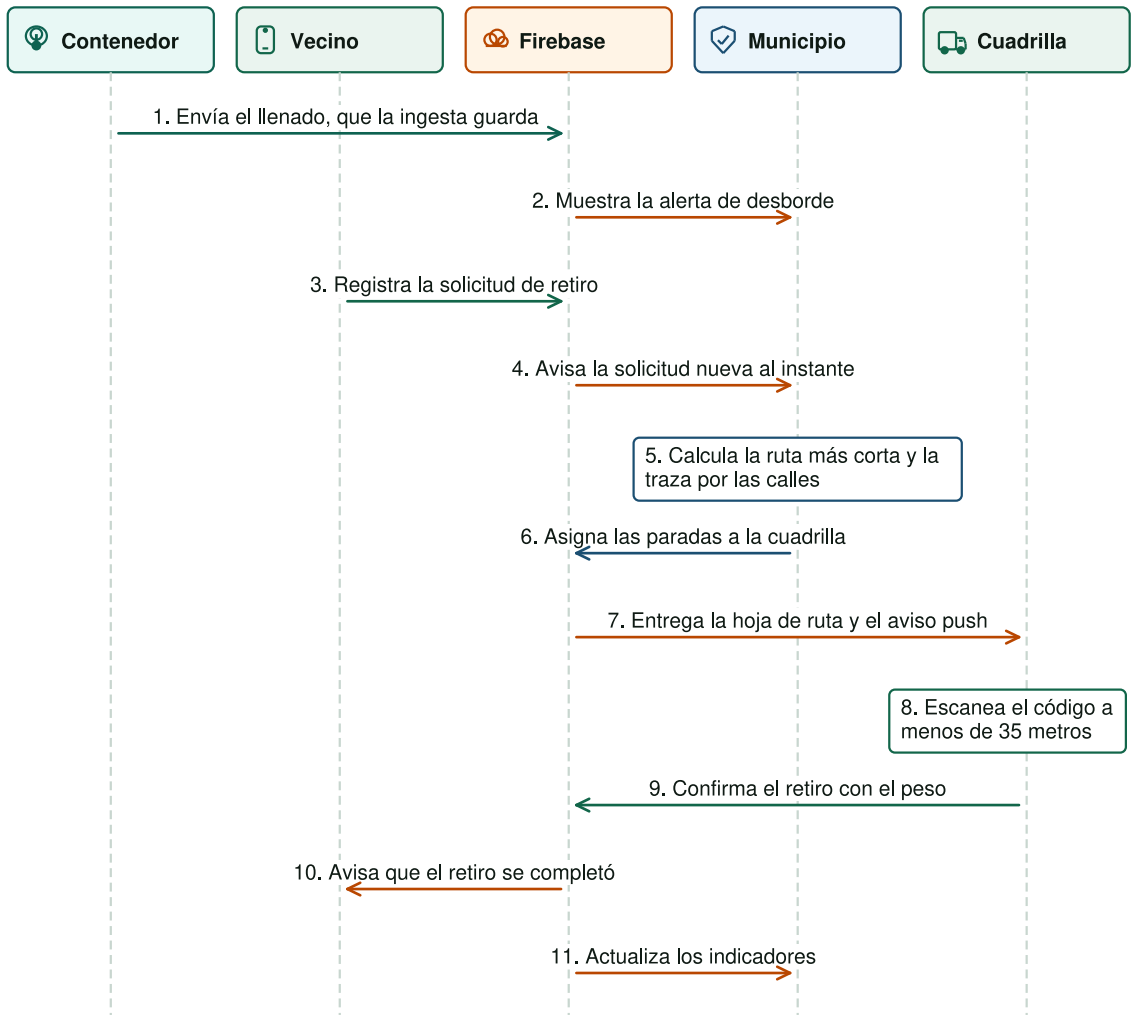
Componentes del sistema y relación entre ellos.

4.2. Flujo de trabajo y secuencia operativa

El circuito de recolección coordina al vecino, a la central municipal y a la cuadrilla en la calle. La secuencia va desde la lectura del sensor hasta el cierre del pedido.

El procedimiento se desarrolla en cinco etapas.

- El vecino elige la categoría del residuo, indica la cantidad y confirma su ubicación. La solicitud queda registrada con estado pendiente.
- El panel municipal recibe la solicitud en el momento. Las solicitudes elegidas se agrupan y el sistema calcula el recorrido más corto.
- Al confirmar la ruta, la cuadrilla asignada recibe las paradas en el orden del turno y un aviso en el celular.
- En cada punto, la cuadrilla escanea la placa del contenedor o ingresa su código. El sistema exige estar a menos de 35 metros del lugar.
- Con el peso cargado, el retiro se cierra y el vecino recibe el aviso de que su solicitud se completó.



Secuencia del circuito de alerta, solicitud, asignación y recolección.

4.3. Modelo de persistencia y datos en Cloud Firestore

Cloud Firestore guarda los datos porque sincroniza los cambios en tiempo real entre todos los dispositivos y permite definir reglas de seguridad por rol. La información se organiza en colecciones.

- Solicitudes. Reúnen el vecino, el tipo de residuo, la cantidad, la ubicación, la fecha, el estado y la cuadrilla asignada.
- Incidentes. Registran cada problema reportado en la calle, su gravedad, su ubicación y su estado.
- Notificaciones. Guardan las campañas que el municipio difunde a la comunidad.
- Telemetría. Contiene el llenado, la temperatura y la batería de cada contenedor.
- Dispositivos. Asocian cada celular que aceptó los avisos con su usuario.
- Roles. Indican qué cuentas de Google son autoridad o cuadrilla, y solo las lee su propio titular.

Los vecinos solo crean y consultan sus propias solicitudes. Las autoridades leen y modifican todo. Las cuadrillas únicamente cambian los campos propios de su tarea, como el estado de la parada y el peso. El catálogo de ecopuntos y las rutas calculadas se mantienen en la aplicación.

4.4. Algoritmo de optimización de rutas de recolección

Ordenar las paradas de una cuadrilla equivale al problema del viajante, que consiste en hallar el recorrido más corto que pasa por todos los puntos. Su solución exacta es demasiado lenta para calcularse al instante con muchas paradas, por lo que se combinan dos métodos que dan buenos resultados rápidamente.

- El primero parte de la base y visita siempre la parada pendiente más cercana a la posición actual.
- El segundo revisa ese recorrido e invierte tramos mientras eso acorte la distancia total. En 1.000 conjuntos de paradas elegidas al azar dentro de la ciudad, redujo el recorrido en promedio un 5,0% con 5 paradas, un 7,1% con 8 y un 7,4% con 12, sin empeorarlo nunca.

Las comparaciones usan la distancia en línea recta sobre la esfera terrestre. Una vez fijado el orden, el servicio de ruteo OSRM traza el recorrido por las calles reales de San Francisco y calcula los kilómetros que recorrerá el camión.

4.5. Sensores en los contenedores y protocolo de comunicación

Cada contenedor inteligente incorpora un microcontrolador de bajo consumo con un sensor ultrasónico, que mide qué tan lleno está, y una sonda de temperatura. La comparación de cuatro protocolos de envío llevó a elegir MQTT.

MQTT desacopla a quien mide de quien consume el dato, y su cabecera mínima alarga la vida de la batería. Tres mecanismos garantizan la llegada de la información.

- La entrega con confirmación asegura que el intermediario reciba cada lectura y cada alarma.
- Un aviso de desconexión, que el intermediario publica por su cuenta, indica si un sensor deja de responder por falta de batería o de señal.
- Un puerto adicional para navegadores permite que el panel municipal reciba los eventos sin servidores intermedios.

Los canales de mensajes se organizan por contenedor y separan las mediciones periódicas, las alarmas urgentes y las órdenes del municipio, como cambiar la frecuencia de muestreo. Cada mensaje informa el identificador del contenedor, la hora, la distancia medida, el porcentaje de llenado, la temperatura y la batería.

Un servicio de ingesta, alojado en el clúster junto a Mosquitto, toma las lecturas y actualiza el llenado en Firestore. Si un contenedor supera el 80%, genera una alerta prioritaria y suma su parada a la ruta de la cuadrilla. El prototipo reproduce este circuito con sensores simulados que escriben sus lecturas en la nube.

El siguiente cuadro resume la comparación que fundamenta la elección.

Protocolo	Funcionamiento	Consumo de datos	Decisión
MQTT	Los sensores publican mensajes en canales y los interesados se suscriben, sobre una conexión permanente	Cabecera mínima de 2 bytes	Elegido por su bajo consumo, la entrega confirmada y la reconexión automática
CoAP	Consulta y respuesta binaria sobre mensajes sueltos	Cabecera de 4 bytes	Descartado porque los navegadores no lo soportan
HTTP	Cada lectura es una consulta que espera respuesta	Más de 500 bytes por mensaje	Descartado por el consumo de batería
LoRaWAN	Radio de largo alcance con envío en un solo sentido	Variable según la distancia	Complementa a MQTT en zonas sin cobertura celular mediante una pasarela

Comparación de protocolos de comunicación para los sensores.

5. PROTOTIPO FUNCIONAL

5.1. Módulo de autenticación, perfiles y control de acceso

Firebase Authentication identifica a cada persona con su cuenta de Google. No existen contraseñas propias, y cada sesión recibe una credencial firmada que Cloud Firestore verifica en sus reglas de seguridad.

El rol de cada cuenta lo asigna el servidor. Las cuentas de la autoridad y de la cuadrilla figuran en una lista privada, y cualquier otra cuenta entra como vecino. Hay tres perfiles.

- El vecino pide retiros, consulta ecopuntos y horarios, y acumula puntos ambientales. No accede a funciones operativas.
- La autoridad municipal supervisa el territorio, ve los contenedores en vivo y despacha rutas.
- La cuadrilla consulta su hoja de ruta, escanea códigos QR y registra el peso recolectado.

Cada perfil tiene su propio tablero y no existe un selector de rol en la navegación. Para cambiar de perfil se cierra la sesión y se ingresa con otra cuenta.

El siguiente cuadro resume lo que cada perfil puede hacer. Las reglas de seguridad del servidor lo aplican aunque alguien intente saltear la pantalla.

Acción	Vecino	Autoridad	Cuadrilla
Crear y ver sus propias solicitudes	Sí	Sí	No
Ver las solicitudes de todos los vecinos	No	Sí	Sí
Asignar paradas y despachar rutas	No	Sí	No
Confirmar la recolección y cargar el peso	No	Sí	Sí
Reportar un incidente	Sí	Sí	Sí
Marcar un incidente como resuelto	No	Sí	Sí
Enviar una campaña a la comunidad	No	Sí	No
Publicar lecturas de los sensores	No	Sí	No

Permisos de cada perfil sobre los datos del sistema.

5.2. Módulo ciudadano

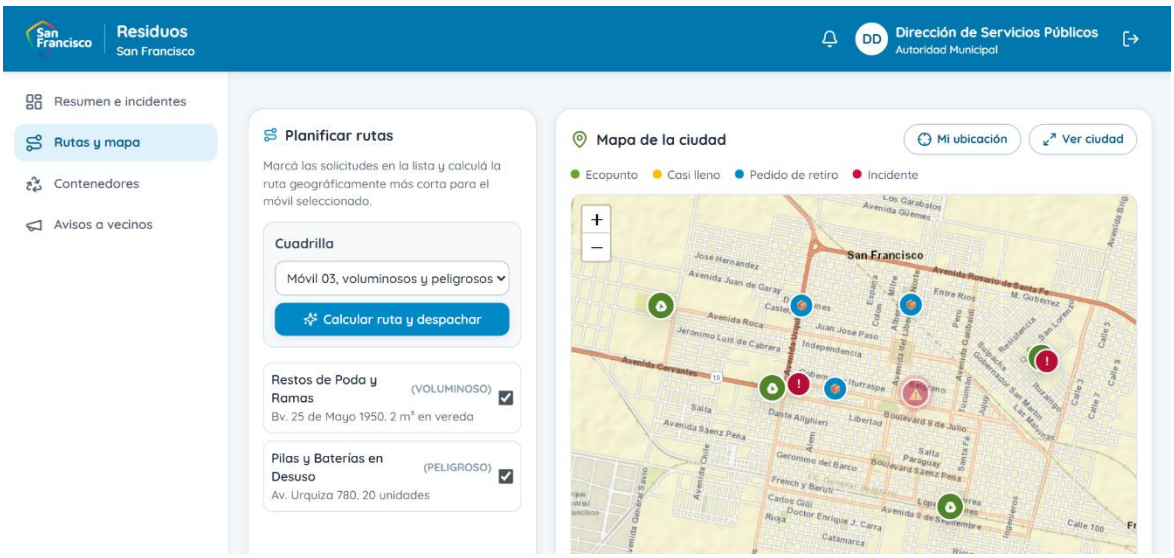
La aplicación del vecino evita formularios largos que desalienten la participación y se adapta al celular.

- La clasificación ofrece reciclables, voluminosos y peligrosos, con subcategorías como papel y cartón, restos de poda, pilas y baterías.
- La solicitud de retiro pide la cantidad aproximada, la dirección, la ubicación en el mapa y la fecha deseada.
- El mapa muestra los ecopuntos con su dirección, su horario de atención, los materiales que reciben y su nivel de llenado. Debajo, cada punto figura como disponible, casi lleno o lleno.
- Una tabla informa por zona los días y horas de recolección domiciliaria, de reciclables y de retiros especiales, y resalta la zona del vecino.
- El centro de avisos muestra el historial de pedidos con su avance y recibe en el momento las campañas municipales. Con la aplicación cerrada, los avisos llegan como notificación del teléfono.

5.3. Panel de control para autoridades municipales

El panel concentra las herramientas de supervisión y despacho de la Dirección de Servicios Públicos.

- Los indicadores resumen las solicitudes pendientes, las cuadrillas en la calle, los incidentes abiertos y los ecopuntos activos.
- La vista de contenedores muestra el llenado, la temperatura y la batería de cada uno, con actualización en vivo, y permite calibrar los sensores.
- El despachador calcula el recorrido óptimo de las solicitudes marcadas, lo traza por las calles y lo asigna a la cuadrilla elegida, que recibe el aviso de inmediato.
- La consola de incidentes lista los problemas reportados, los clasifica por gravedad y permite marcarlos como resueltos.
- Las campañas se redactan en el panel y llegan en vivo a vecinos y cuadrillas.



Panel municipal en una pantalla de escritorio.

En escritorio, el panel organiza sus secciones en un menú lateral y cada una se abre en su propia pantalla. En el celular esas mismas secciones se reúnen en una barra inferior.

5.4. Módulo operativo para personal de cuadrilla

La aplicación del recolector se diseñó para usarse con una mano y en movimiento, con textos legibles a pleno sol.

- La hoja de ruta presenta las paradas del camión en el orden óptimo, el recorrido por calles con sus kilómetros, un resumen del turno y una marca de urgencia en los contenedores casi llenos.

- El escáner lee la placa de cada contenedor con la cámara o acepta el código ingresado a mano, y rechaza los que no figuran en el registro municipal de contenedores.
- Al tocar Confirmar en una parada se abre una ventana con el código, el peso y las observaciones, sin salir de la hoja de ruta.
- El reporte de problemas se completa con dos toques y alerta de inmediato a la central.

Antes de cerrar una parada, la aplicación comprueba que el código corresponda a un punto o a una solicitud existente y que el teléfono esté a menos de 35 metros del lugar. Si la distancia es mayor, pide una confirmación expresa y deja registrado que la recolección se hizo fuera del radio.

5.5. Plataforma de cartografía y geolocalización

El mapa se construye con la biblioteca Leaflet y las teselas de Esri, que ofrecen un plano urbano nítido, sin marcas de agua ni cargos por uso. Los recorridos por calles provienen de OSRM, un servicio de ruteo abierto. Si no respondiera, el mapa mantiene el trazo en línea recta.

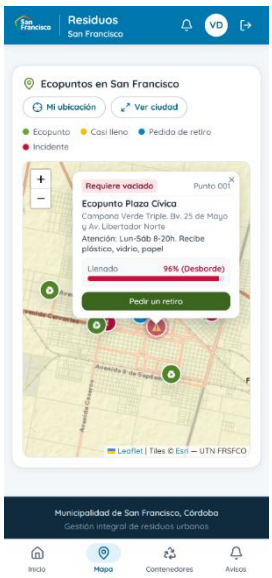
El mapa se centra en San Francisco y distingue por color los ecopuntos, los contenedores casi llenos, los pedidos de retiro y los incidentes. Al tocar un ecopunto se ven su horario, los materiales que recibe y su llenado. Un botón centra la vista en la ubicación del teléfono.

5.6. Capturas de la interfaz en ejecución

Las capturas muestran los perfiles del vecino y de la cuadrilla en el celular.



Portal del vecino.



Horario y llenado de un ecopunto.



Hoja de ruta de la cuadrilla.

El vecino ve solo sus propias solicitudes, con su estado y la cuadrilla asignada, y al tocar un ecopunto consulta su horario y su llenado. La cuadrilla trabaja en una sola pantalla, con el resumen del turno, las paradas en orden y el mapa de su recorrido.

6. ESQUEMA DE ESCALABILIDAD Y MONITOREO

6.1. Contenerización de servicios con Docker

La aplicación se empaqueta en una imagen Docker mínima, basada en el servidor Nginx sobre una distribución mínima de Linux y sin privilegios de administrador. La imagen es liviana, funciona de la misma manera en cualquier servidor y expone pocos puntos de ataque.

Atributo	Imagen estándar de Nginx	Imagen del proyecto	Beneficio
Tamaño en disco	243 MB	29,7 MB	Reduce un 88% la descarga y acelera el inicio de cada réplica
Usuario de ejecución	Administrador del sistema	Usuario sin privilegios	Limita el daño si alguien vulnera el contenedor
Contenido	Servidor genérico con configuración por defecto	Solo la aplicación y su configuración de seguridad	Reduce la superficie de ataque
Tiempo hasta responder	1,0 s	0,8 s	Permite sumar réplicas rápidamente ante picos de uso

Comparación entre la imagen estándar y la imagen del proyecto.

El contenedor incluye controles automáticos de salud. Uno verifica que el servidor responda y otro indica cuándo puede recibir tráfico, de modo que Kubernetes reinicie cualquier réplica que falle.

6.2. Orquestación y balanceo con Kubernetes

Kubernetes ejecuta las réplicas de la aplicación y las separa del acceso desde internet. La configuración se verificó en un clúster local con la imagen del proyecto.

- La puerta de entrada del clúster, basada en Nginx, recibe el tráfico, exige conexión cifrada y limita a 50 peticiones por segundo a cada dirección para evitar abusos.

- El servicio interno ofrece un punto de acceso estable y reparte las peticiones entre las réplicas de forma rotativa.
- La actualización progresiva agrega una réplica nueva antes de retirar la anterior, de modo que los vecinos nunca pierden el servicio durante un cambio de versión.
- La distribución entre servidores separa las réplicas en máquinas distintas, para que la caída de una no detenga la aplicación.

El siguiente cuadro compara las opciones evaluadas para alojar el servicio.

Opción	Ventajas	Desventajas	Decisión
Kubernetes con autoescalado	Reparte la carga, reemplaza las réplicas caídas y escala sin intervención	Requiere administrar un clúster	Elegida para la infraestructura propia del municipio
Servidores virtuales fijos	Configuración simple	Escalar exige trabajo manual y se paga la capacidad ociosa	Descartada
Contenedores administrados por un proveedor	Escalan solos sin administrar un clúster	Atan el servicio a un proveedor y limitan el control de la red	Alternativa si el municipio no administra servidores
Solo Firebase Hosting	Sin servidores que mantener y con distribución mundial	No aloja Mosquitto ni el servicio de ingesta	Usada para publicar el prototipo

Opciones evaluadas para alojar el servicio.

6.3. Autoescalado horizontal de réplicas

El autoescalado ajusta la cantidad de réplicas según el uso de recursos que mide el clúster.

- Cuando el procesador supera el 70% de lo reservado para cada réplica, el controlador agrega instancias.
- El mínimo es de 2 réplicas en horarios de poca actividad y el máximo es de 10 en emergencias o campañas.
- Ante un aumento brusco, puede duplicar las réplicas cada 15 segundos para no perder velocidad de respuesta.
- Al bajar la demanda, espera 300 segundos antes de retirar réplicas, para evitar encendidos y apagados continuos.

La política se comprobó en un clúster local con seis generadores de carga. El uso del procesador se mide sobre lo reservado para cada réplica, por lo que puede superar el 100%.

Tiempo desde el inicio de la carga	Réplicas en funcionamiento	Uso del procesador
0 segundos	2	1%
30 segundos	2	73%
45 segundos	2, y el autoescalado ya pidió 6	491%
60 segundos	6	470%
80 segundos	10	433%
100 segundos	10	145%

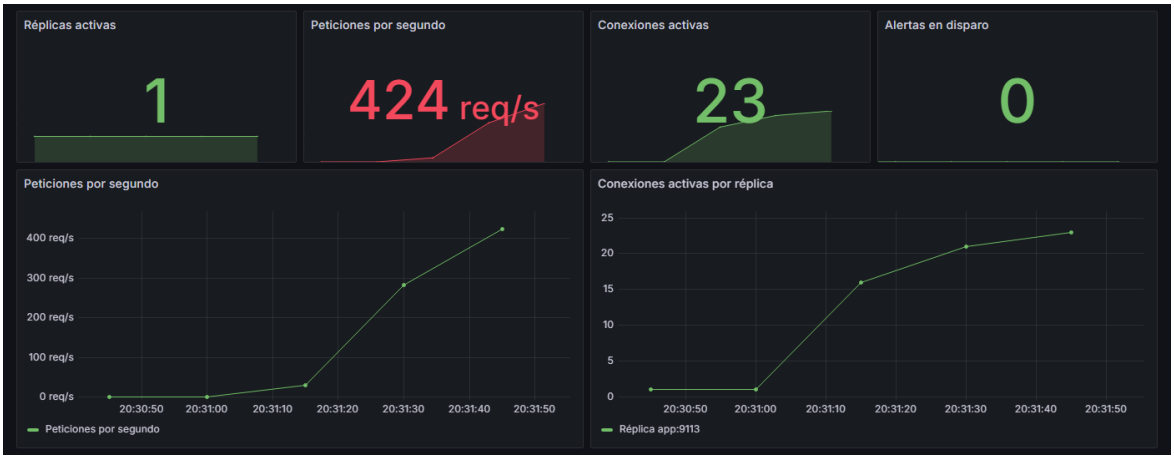
Respuesta del autoescalado ante una carga sostenida.

6.4. Observabilidad y monitoreo con Prometheus y Grafana

Cada réplica incorpora un exportador que publica las métricas del servidor Nginx. Prometheus las recolecta cada 15 segundos y Grafana las presenta en un tablero.

- El tablero muestra las réplicas activas, las peticiones por segundo y las conexiones abiertas, con actualización cada 5 segundos.
- Una alerta crítica se activa si una réplica deja de responder durante 30 segundos.
- Una advertencia se activa si el tráfico supera las 200 peticiones por segundo durante un minuto o si las conexiones abiertas superan las 400.
- Otra advertencia indica que el autoescalado llegó al máximo de 10 réplicas, para evaluar una ampliación.

En la prueba local con 60 clientes durante 60 segundos, el tablero registró un pico de 456 peticiones por segundo. El tráfico superó las 200 por segundo durante menos de un minuto, por lo que la advertencia no llegó a activarse. Al detener el exportador, la alerta de caída se activó a los 45 segundos.



Tablero de Grafana durante la prueba de carga.

7. ESTRATEGIA DE PRUEBAS Y CALIDAD

Las pruebas automáticas recorren cada perfil, comprueban la sincronización entre dispositivos y verifican las reglas de seguridad contra la base publicada. Se ejecutaron sobre la copia local, el contenedor Docker y la aplicación publicada.

Nivel	Herramienta	Qué se verificó	Resultado
Flujos por perfil	Navegador automatizado	Recorrido completo del vecino, la autoridad y la cuadrilla, con cada tablero aislado	Aprobado
Datos en vivo	Tres sesiones simultáneas	Solicitudes, despachos, campañas e incidentes que llegan en vivo a los otros perfiles	Aprobado
Validación de ubicación	Navegador con ubicación simulada	Aceptación dentro de los 35 metros y confirmación expresa fuera de ellos	Aprobado
Seguridad	Consultas directas a la base y emulador	Permisos de cada rol, rechazo sin sesión y asignación de roles por cuenta de Google	Aprobado
Avisos push	Chrome y Firebase Cloud Messaging	Aviso recibido con la aplicación cerrada y baja del dispositivo al cerrar sesión	Aprobado
Rutas	1.000 conjuntos de paradas por tamaño	Mejora del segundo método sobre el primero	Entre 5,0% y 7,4%

Nivel	Herramienta	Qué se verificó	Resultado
Carga	60 clientes durante 60 segundos	Respuesta del contenedor Docker	432 peticiones por segundo, ninguna con error y 95% en menos de 214 ms
Autoescalado	Clúster local y generadores de carga	Aumento de réplicas bajo demanda	De 2 a 10 réplicas en poco más de un minuto
Calidad web	Lighthouse	Accesibilidad, buenas prácticas, buscadores y rendimiento	Accesibilidad 100, prácticas 96, buscadores 92, rendimiento 100 en escritorio y 77 en celular
Sensores	Simulador de contenedores	Lecturas sincronizadas con Firestore	Aprobado

Pruebas realizadas sobre el prototipo y sus resultados.

El rendimiento en celular mejoró de 36 a 77 puntos al compilar los estilos, alojar las bibliotecas en el propio sitio, reducir los íconos de 358 a 34 KB y cargar el lector de códigos solo al escanear.

8. PROBLEMAS ENCONTRADOS Y SOLUCIONES TÉCNICAS

Durante la integración surgieron dificultades que exigieron ajustes en la arquitectura. El cuadro resume cada problema, su causa y la solución aplicada.

Problema	Causa	Solución
El mapa se bloqueaba o mostraba marcas de agua	Los servidores comunitarios rechazaban las peticiones y otro proveedor exigía clave y superponía su marca	Cambio a las teselas de Esri y ajuste de la política de seguridad del servidor
Cabeceras de seguridad ausentes en los archivos estáticos	Nginx no hereda las cabeceras en una sección que define las suyas	Repetición de las cabeceras en la sección de recursos estáticos
Lectura de códigos QR lejos del contenedor	El escáner aceptaba el código sin comprobar dónde estaba el camión	Validación de la ubicación a menos de 35 metros, con confirmación expresa fuera del radio

Problema	Causa	Solución
Riesgo de inyección de código en los listados	Los textos de los vecinos se insertaban en la página sin filtrar	Función que neutraliza los caracteres peligrosos antes de mostrar cualquier texto
Solicitudes que no llegaban a la nube	El ingreso con cuentas locales no creaba una identidad que las reglas aceptaran	Ingreso con Firebase Authentication y cuentas de Google
Cuentas de demostración públicas	Las claves de la autoridad y de la cuadrilla estaban en el código y cualquiera podía usarlas	Roles por cuenta de Google guardados en el servidor y eliminación de las cuentas de prueba
La pantalla de ingreso no aparecía	Datos viejos guardados en el navegador interrumpían el arranque	Descarte de los datos de versiones anteriores y arranque que no depende de ellos
Monitoreo sin métricas	Las réplicas no exponían datos que Prometheus pudiera leer	Exportador de métricas de Nginx en cada réplica y alertas sobre esas métricas

Problemas detectados, causas y soluciones adoptadas.

9. ENTREGABLES Y CUMPLIMIENTO DE CONSIGNAS

El siguiente cuadro relaciona cada requerimiento de la consigna con la funcionalidad que lo cubre.

Requerimiento	Cómo se cumple
Registro de residuos por tipo y volumen	Formulario con categoría, subcategoría y cantidad estimada
Retiro programado de reciclables, voluminosos y peligrosos	Solicitud con dirección, ubicación en el mapa y fecha deseada
Ecopuntos, horarios y campañas	Mapa con dirección, horario, materiales y llenado de cada punto, tabla de recolección por zona y avisos de campañas
Puntos de recolección en tiempo real	Panel municipal con contenedores y solicitudes que se actualizan al instante
Rutas optimizadas para las cuadrillas	Recorrido más corto calculado con dos métodos combinados, trazado por las calles y asignado a una cuadrilla
Telemetría, alertas de desborde e incidentes	Llenado, temperatura y batería de cada contenedor, alerta sobre el 80% y consola de incidentes

Requerimiento	Cómo se cumple
Hoja de ruta, códigos QR y reportes viales	Paradas ordenadas, escaneo con validación a 35 metros y reporte de problemas en dos toques
Mapeo y geolocalización	Mapa con Leaflet y Esri, ubicación del teléfono y recorridos por calles con OSRM
Notificaciones para vecinos y cuadrillas	Avisos en vivo y notificaciones push que llegan con la aplicación cerrada
Contenedores Docker y Kubernetes	Imagen de 29,7 MB, réplicas, servicio interno e ingreso cifrado
Monitoreo y autoescalado	Prometheus, Grafana y alertas, con autoescalado de 2 a 10 réplicas comprobado bajo carga

Cumplimiento de los requerimientos de la consigna.

El proyecto reúne los siguientes entregables, que permiten reproducir la solución completa.

- La aplicación web móvil con sus tres perfiles, el cálculo de rutas, el lector de códigos QR, los avisos push y el funcionamiento sin conexión.
- La imagen Docker y los archivos de configuración de Kubernetes, con ingreso cifrado, autoescalado y distribución entre nodos.
- Las reglas de seguridad de Firestore por rol y sus índices.
- El monitoreo con Prometheus, sus alertas y el tablero de Grafana.
- El emisor de avisos push y el simulador de sensores, con el servicio de ingesta y el intermediario Mosquitto.
- La documentación técnica, con los diagramas, la justificación de las decisiones y los resultados de las pruebas.

La aplicación publicada se encuentra en https://sf-residuos-urbanos-utn.web.app y su circuito completo se recorre con tres ventanas.

- Vecino. Con cualquier cuenta de Google, registra una solicitud de retiro voluminoso y revisa el mapa, los horarios y los contenedores.
- Autoridad. Con una cuenta habilitada por los autores a pedido del docente, ve llegar la solicitud, elige la cuadrilla y despacha la ruta, que se traza por las calles. También envía una campaña.
- Cuadrilla. Con otra cuenta habilitada del mismo modo, recibe la parada, ingresa el código del contenedor y confirma el peso.
- Avisos. Una vez aceptadas las notificaciones, el emisor de avisos envía un mensaje que llega al celular con la aplicación cerrada.

10. INNOVACIÓN Y PARTICIPACIÓN CIUDADANA

Cuatro mejoras potencian la eficiencia del servicio y la participación de los vecinos.

- Un sistema de incentivos acredita puntos por cada kilo de reciclables entregado, canjeables por bonificaciones en tasas municipales. El prototipo ya muestra el aporte de cada vecino.
- Una predicción del llenado estima, con el historial de cada sensor, cuándo se completará cada contenedor y lo suma a la ruta antes de que desborde.
- Un mapa colaborativo permite denunciar basurales con foto y ubicación, para que las cuadrillas intervengan con mayor rapidez.
- Un ranking mensual de barrios publica los kilos reciclados por zona y premia a los que más mejoran.

11. CONCLUSIONES

El prototipo muestra que un servicio público esencial puede modernizarse con arquitecturas móviles y servicios de nube.

- Una aplicación web instalable con Firebase y servicios en Kubernetes aporta agilidad de desarrollo sin perder robustez.
- El trazado de rutas con dos métodos combinados y la validación en la calle reducen los kilómetros recorridos y dejan registro del trabajo operativo.
- El autoescalado llevó el sistema de 2 a 10 réplicas ante una carga sostenida, y el tablero de monitoreo permitió observar el pico y detectar la caída de una réplica.
- La ubicación, los avisos en tiempo real y el seguimiento de cada solicitud fomentan el compromiso de los vecinos con el cuidado ambiental.

12. BIBLIOGRAFÍA

- Bono, J. P. (2026). *Apuntes de Cátedra: Modelos de Arquitecturas basados en Cloud Computing y Escalado de Aplicaciones*. UTN Facultad Regional San Francisco.
- Google. (2026). *Cloud Firestore documentation*. Firebase.
- Google. (2026). *Firebase Cloud Messaging documentation*. Firebase.
- Grafana Labs. (2026). *Grafana documentation*. Grafana Labs.
- Kubernetes Authors. (2026). *Horizontal Pod Autoscaling*. Kubernetes Documentation.

- Leaflet. (2026). *Leaflet API reference*. Leaflet.
- Newman, S. (2021). *Building Microservices: Designing Fine-Grained Systems* (2nd ed.). O'Reilly Media.
- OASIS. (2019). *MQTT Version 5.0*. OASIS Standard.
- Project OSRM. (2026). *OSRM API documentation*. Project OSRM.
- Prometheus Authors. (2026). *Prometheus documentation*. Cloud Native Computing Foundation.
- van Drongelen, M., Dennis, A., Garabedian, R., Gonzalez, A. y Krishnaswamy, A. (2017). *Lean Mobile App Development: Apply Lean startup methodologies to develop successful iOS and Android apps*. Packt Publishing.